async / await

10 min read Updated 17 Jul 2026 Topic 17 of 23 Bookmark Completed

`async` / `await` is syntactic sugar over promises: it lets you write asynchronous code that reads top-to-bottom like synchronous code, while still being non-blocking under the hood. Interviewers lean on it hard because it exposes whether you actually understand promises and the event loop — the single most common real-world bug it produces (awaiting independent calls one-by-one) is a favourite "spot the performance problem" question. Master three things and you can answer almost anything they throw: an `async` function always returns a promise, `await` pauses that function until a promise settles, and `await` yields control back to the event loop's microtask queue.

`async` always returns a promise; `await` unwraps one

Marking a function `async` does two things: any value you `return` is wrapped in a resolved promise, and any exception you `throw` becomes a rejected promise. So the caller always gets a promise back, never a bare value — you consume it with `.then` / `.catch` or another `await`.

JS Copy

```
async function f() { return 42; }
const r = f();
console.log(r instanceof Promise); // ⇒ true
f().then((v) ⇒ console.log(v));    // ⇒ 42

async function g() { throw new Error('boom'); }
g().catch((e) ⇒ console.log('caught', e.message)); // ⇒ caught boom
```

`await` is the mirror image: it takes a promise, suspends the surrounding `async` function until that promise settles, and evaluates to the resolved value (or throws the

rejection reason). It is purely sugar over `.then` — the two versions below are equivalent.

JS Copy

```
function fetchUser() { return Promise.resolve({ name: 'Ada' }); }

async function withAwait() {
  const user = await fetchUser();
  return user.name.toUpperCase();
}

function withThen() {
  return fetchUser().then((user) => user.name.toUpperCase());
}

withAwait().then((v) => console.log('await:', v)); // => await: ADA
withThen().then((v) => console.log('then:', v));   // => then: ADA
```

The `await` version is not doing anything the `.then` version can't — it's the same promise chain, just flattened so you don't nest callbacks. This equivalence is the answer to "what is async/await really?": **it's a compiler transform into promise `.then` chains**, nothing more.

Error handling: try/catch vs .catch, and rethrowing

Because a rejected `await` *throws*, you handle errors with ordinary `try/catch` instead of `.catch`. Inside a `try`, an awaited rejection jumps straight to the `catch` block — including errors from calls several `await`s deep. A common pattern is to log-and-rethrow so the current layer records the failure but the caller still sees it.

JS Copy

```
async function load() {
  try {
    const res = await Promise.reject(new Error('network down'));
    return res;
  } catch (err) {
    console.log('logging:', err.message);
    throw err; // rethrow: caller still sees the failure
```

```
load().catch((e) => console.log('caller saw:', e.message));
// => logging: network down
// => caller saw: network down
```

`try/catch` and `.catch` are interchangeable in what they catch, but `try/catch` also wraps synchronous throws in the same block, and reads more naturally when you have several sequential awaits. The trap (covered below) is that `try/catch` only catches promises you actually `await` inside the block.

Sequential vs parallel — the performance bug interviewers love

This is the highest-value part of the topic. When you `await` calls one after another, each one waits for the previous to finish — even if they don't depend on each other. If the calls are independent, that's wasted time: start them all first, then await them together with `Promise.all`.

JS Copy

```
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

// SEQUENTIAL: each await waits for the previous — total ~300ms
async function sequential() {
  const t0 = Date.now();
  const a = await delay(100, 'a');
  const b = await delay(100, 'b');
  const c = await delay(100, 'c');
  console.log('sequential:', [a, b, c], Date.now() - t0, 'ms');
}

// PARALLEL: start all three, then await together — total ~100ms
async function parallel() {
  const t0 = Date.now();
  const [a, b, c] = await Promise.all([
    delay(100, 'a'),
    delay(100, 'b'),
```

```
}

(async () => { await sequential(); await parallel(); })();
// => sequential: [ 'a', 'b', 'c' ] 304 ms
// => parallel:   [ 'a', 'b', 'c' ] 101 ms
```

Same result, ~3x the latency in the sequential version. The key insight for the interview: `Promise.all` doesn't make things run "in parallel" in a threading sense — JS is single-threaded — it just lets the three timers (or network requests) be *in flight at the same time*. The fix is to kick off the async work before the first `await`. Note `Promise.all` is fail-fast: if any promise rejects, the whole thing rejects immediately with that reason (use `Promise.allSettled` when you want every result regardless).

Loops: `for ... of` when order matters, `.map` + `Promise.all` when it doesn't

`await` inside a `for ... of` loop runs iterations strictly in sequence — use this when each step depends on the last, or when you must not overload a downstream service. When the iterations are independent, map them to promises and await them together instead.

JS Copy

```
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

// Order matters: process steps in sequence with for...of
async function inOrder(steps) {
  const out = [];
  for (const s of steps) {
    out.push(await delay(20, s.toUpperCase()));
  }
  return out;
}

// Order does NOT matter: fire all, collect with Promise.all
```

```
async function anyOrder(items) {
  return Promise.all(items.map((i) => delay(20, i.toUpperCase())));
}

(async () => {
  console.log('inOrder:', await inOrder(['a', 'b', 'c']));   // => [ 'A',
  console.log('anyOrder:', await anyOrder(['a', 'b', 'c'])); // => [ 'A',
})();
```

Two things worth knowing for the interview. First, `Promise.all` **preserves input order** in its result array regardless of which promise settles first, so the `anyOrder` output is still `A, B, C`. Second, `array.forEach(async ...)` does *not* wait for anything — `forEach` ignores the returned promises, so the loop finishes before any async work does. Use `for ... of` or `map` + `Promise.all`, never `forEach`, with async callbacks.

For a stream of async values, `for await ... of` iterates an async iterable, awaiting each yielded value in turn:

JS Copy

```
async function* pages() {
  yield 'page1';
  yield 'page2';
  yield 'page3';
}
(async () => {
  for await (const p of pages()) {
    console.log('got', p); // => got page1 / got page2 / got page3
  }
})();
```

Top-level await and the event loop

In ES modules you can use `await` at the top level, outside any `async` function — a module with top-level `await` becomes async, and modules importing it wait for it to finish evaluating before their own code runs. It's handy for conditional imports or async config, but it delays every dependent module, so keep it lean.

```
// inside an ESM file (.mjs or "type": "module")
const value = await Promise.resolve('module ready');
console.log(value); // ⇒ module ready
```

Now the mechanic that ties everything together: `await` **yields to the microtask queue.** When execution hits an `await`, the `async` function suspends and returns control to the caller; the code *after* the `await` is scheduled as a microtask that runs only once the current synchronous run of code (and the awaited promise) has completed. This is why the lines below print in a surprising order.

JS Copy

```
console.log('1 sync start');

async function a() {
  console.log('2 before await');
  await null;                 // suspends here, resumes as a microtask
  console.log('5 after await');
}

a();                          // await-continuation queued first …
console.log('3 sync after a()');
Promise.resolve().then(() ⇒ console.log('6 promise .then'));
console.log('4 end of sync');
// ⇒ 1 sync start
// ⇒ 2 before await
// ⇒ 3 sync after a()
// ⇒ 4 end of sync
// ⇒ 5 after await
// ⇒ 6 promise .then
```

Everything up to the `await` (`1`, `2`) runs synchronously. At `await null` the function suspends and control returns to the top level, which finishes its synchronous work (`3`, `4`). Only then does the microtask queue drain: the `await` continuation (`5`) was enqueued before the `.then` callback (`6`), so it runs first. The takeaway for "what does this print?" questions: an `async` function runs synchronously up to its first `await`, and

Common interview questions

- **What does an `async` function return?** Always a promise. A returned value becomes a resolved promise; a thrown error becomes a rejected one. Even `async () => {}` returns a promise that resolves to `undefined`.
- **Is `async/await` just syntactic sugar? Over what?** Yes — over promises and `.then` / `.catch` chains. It doesn't add capability; it flattens the callback nesting and lets you use `try/catch` and normal control flow.
- **How do you run independent async calls concurrently?** Start them without awaiting each individually, then `await Promise.all([ ... ])`. Awaiting them one at a time serializes independent work and multiplies the latency.
- **Difference between `Promise.all`, `Promise.allSettled`, `Promise.race`?** `all` resolves with all values or rejects on the first rejection (fail-fast). `allSettled` waits for every promise and reports each as fulfilled/rejected. `race` settles as soon as the first promise settles (resolve or reject).
- **Does `await` block the thread?** No. It suspends only the current `async` function and yields to the event loop; other tasks, timers, and I/O keep running. "Blocking" would freeze the whole runtime — `await` does the opposite.
- **When does the code after an `await` run?** As a microtask, after the current synchronous stack unwinds and the awaited promise settles — before any `setTimeout` callback.

Gotchas / traps

- **Awaiting sequentially by accident.** `const a = await x(); const b = await y();` runs `y` only after `x` finishes. If `x` and `y` are independent, you just doubled the latency. Fix: `const [a, b] = await Promise.all([x(), y()])`. This is the #1 async/await bug in real code and a staple interview question.
- **Forgetting `await`.** Without it you hold the *promise*, not the value — and the bug is often silent until you call a method on it.

```
async function getName() { return 'Ada'; }
async function bug() {
  const name = getName();                // forgot await — name is a Promis
  console.log(name.toUpperCase());       // Promise has no toUpperCase()
}
bug().catch((e) => console.log('TypeError:', e.message));
// => TypeError: name.toUpperCase is not a function
```

- `return` vs `return await` inside `try`. `return somePromise` in a `try` block returns the promise *without awaiting it*, so the function has already left the `try` — the local `catch` never fires and the rejection escapes to the caller. `return await somePromise` awaits inside the `try`, so the `catch` catches it. If you have a surrounding `try/catch`, prefer `return await`.

JS Copy

```
function fail() { return Promise.reject(new Error('inner')); }

async function plainReturn() {
  try { return fail(); }                     // NOT awaited — catch can't
  catch (e) { return 'caught by plainReturn'; }
}
async function returnAwait() {
  try { return await fail(); }               // awaited — catch fires
  catch (e) { return 'caught by returnAwait'; }
}
(async () => {
  console.log('plainReturn:', await plainReturn().catch((e) => 'ESCAPED: '
  console.log('returnAwait:', await returnAwait());
})();
// => plainReturn: ESCAPED: inner
// => returnAwait: caught by returnAwait
```

- `try/catch` scope. A `try/catch` only catches promises you `await` *inside* the block. If you start a promise inside the `try` but await it later (or never), the `catch` won't fire for its rejection. Await it where you want the error handled.

or `Promise.all(arr.map( ... ))` (concurrent).

- **Unhandled rejections.** A rejected promise with no `.catch` / `try` anywhere triggers an `unhandledRejection` (and can crash Node). Always terminate a chain with error handling.

QUICK CHECK — no marks, just for you.

1. What does this log, and in what order?

JAVASCRIPT Copy

```
console.log('A');
async function run() {
  console.log('B');
  await null;
  console.log('C');
}
run();
console.log('D');
```

- ○ A B C D
- ○ A D B C
- ○ A B D C
- ○ A B C

2. The three delays settle in the order b, c, a. What does this log?

JAVASCRIPT Copy

```
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}
async function main() {
  const [a, b, c] = await Promise.all([delay(30, 'a'), delay(10, 'b'), delay
  console.log([a, b, c]);
}
main();
```

- ◯ `[ 'a', 'b', 'c' ]`
- ◯ `[ 'b', 'c', 'a' ]`
- ◯ `[ 'a', 'c', 'b' ]`
- ◯ `[ 'c', 'b', 'a' ]`

Check answer

3. What does this log?

JAVASCRIPT Copy

```
async function f() { return 42; }
const r = f();
console.log(r instanceof Promise);
console.log(typeof r.then);
```

- ◯ `false`
  `undefined`

- false
  function
- true
  function

Check answer

Previous
Promises Deep Dive

Next
Callbacks, Generators & Iterators

LEARN MORE

ARTICLES & DOCS

- async function (MDN)
  MDN
- Using promises (MDN)
  MDN

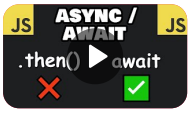
VIDEOS



- async/await simply explained
  YouTube



- async/await vs promises
  YouTube



- async/await is easy!
  YouTube

Hand-picked references for this lesson.

JavaScript Interview Kit